



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LAB REPORT

Title: OWASP Top 10 Risk Assessment and Secure Application Design
for a Vulnerable Web Platform

Student Name: Ahmad Zubairu Garba

Student ID: 2025/FWSD/11463

Course: Web Application Security Essentials

Course Code: BVWS102

Instructor: AHMED BUKAR

19/NOV/2025

WEEK 3 LABORATORY REPORT

TASK 1: UNDERSTAND OWASP TOP 10

Define OWASP and explain why the OWASP Top 10 is important in web application security

The **Open Web Application Security Project (OWASP)** is a non-profit foundation dedicated to improving the security of software. It provides unbiased, practical information about application security.

The **OWASP Top 10** is a standard awareness document for developers and security professionals. It represents a broad consensus regarding the most critical security risks facing web applications today. It is crucial because it helps organizations prioritize their security efforts by focusing on the most common and dangerous vulnerabilities. By addressing these 10 risks, development and security teams can significantly reduce the likelihood of a successful cyberattack against their applications.

OWASP Top 10 Risks and Real-World Examples

1. A01: Broken Access Control:

- **Vulnerability:** Occurs when restrictions on what authenticated users are allowed to access are not properly enforced. This can lead to unauthorized information disclosure, modification, or destruction of data.
- **Breach Example:** The **Equifax (2017)** breach was linked to a lack of proper access controls on a database file, leading to the exposure of personal data for over 147 million people.

2. A02: Cryptographic Failures:

- **Vulnerability:** Pertains to sensitive data exposure due to improper data protection during transit or at rest. This includes poor key management, using deprecated algorithms, or failing to encrypt all necessary data.
- **Breach Example:** In the **Ashley Madison (2015)** breach, user passwords and other sensitive information were not properly encrypted, allowing attackers to access and leak the data.

3. A03: Injection:

- **Vulnerability:** Occurs when untrusted data is sent to an interpreter as part of a command or query (e.g., SQL, OS Command, LDAP). The interpreter executes the malicious command, often to access unauthorized data.

- **Breach Example:** The **TalkTalk (2015)** breach was caused by a SQL Injection attack that allowed hackers to access the personal data of over 150,000 customers.

4. A04: Insecure Design:

- **Vulnerability:** A new category focusing on application-level flaws and design issues rather than just coding or implementation errors. It highlights the failure to implement security-by-design principles and robust threat modeling.
- **Breach Example:** Many successful attacks exploit fundamental **design flaws** (e.g., poorly designed logic for multi-factor authentication enrollment or password recovery).

5. A05: Security Misconfiguration:

- **Vulnerability:** Happens when security settings are not properly configured across the application stack (e.g., server, database, framework). This includes using default credentials, leaving unnecessary services enabled, or incorrect HTTP headers.
- **Breach Example:** The **Capital One (2019)** breach was facilitated by a misconfigured Web Application Firewall (WAF) that allowed an attacker to access data from over 100 million customers.

6. A06: Vulnerable and Outdated Components:

- **Vulnerability:** Involves flaws in components, libraries, frameworks, and other modules used by the application. If these components are not patched or are running outdated versions, they present a known, exploitable security risk.
- **Breach Example:** The flaw exploited in the **Equifax (2017)** breach was a known vulnerability in the Apache Struts framework, which the company failed to patch.

7. A07: Identification and Authentication Failures:

- **Vulnerability:** Occurs when an application incorrectly manages user identification, authentication, and session management. This can allow attackers to compromise passwords, keys, or session tokens, or exploit weak log-in logic.
- **Breach Example:** The **Marriott/Starwood (2018)** breach involved compromised system credentials, which allowed unauthorized access to the guest reservation database.

8. A08: Software and Data Integrity Failures:

- **Vulnerability:** Focuses on flaws where the application relies on integrity checks (or lack thereof) for critical data, updates, or CI/CD pipelines. This includes insecure deserialization.
- **Breach Example:** The **SolarWinds (2020) Supply Chain Attack** involved attackers injecting malicious code into the trusted software build and update process, compromising the integrity of the software.

9. A09: Security Logging and Monitoring Failures:

- **Vulnerability:** Occurs when an application lacks sufficient logging, monitoring, and alerting to detect, escalate, and respond to active breaches in a timely manner.
- **Breach Example:** During the **Target (2013)** breach, security systems detected the malicious activity, but the alerts were ignored, leading to a massive data breach.

10. **A10: Server-Side Request Forgery (SSRF):**

- **Vulnerability:** Flaws where a web application fetches a remote resource without validating the user-supplied URL. An attacker can use this to force the application to send a crafted request to an unexpected destination, often an internal network.
- **Breach Example:** Many real-world exploits leverage SSRF to access **cloud metadata endpoints** (like the AWS Instance Metadata Service) to obtain temporary credentials for the cloud environment.

TASK 2: APPLICATION RISK MAPPING (PAYX SECURE)

Five Critical Risks for PayX Secure

1. Injection (A03)

- **How it Could Exist:** If the platform's backend uses unsanitized user input to construct SQL queries for functions like retrieving transaction history or updating an account balance.
- **Attacker Exploitation:** An attacker could enter a malicious string (e.g., ' OR 1=1 --) into an input field, which alters the SQL query to bypass authentication or retrieve data from unauthorized tables.
- **Potential Business Impact:** Massive data breach (exposing customer records and financial details) leading to regulatory fines and severe reputational damage.

2. Identification and Authentication Failures (A07)

- **How it Could Exist:** The platform might have weak password policies, use easily guessable session tokens, or lack **rate limiting** on login attempts.
- **Attacker Exploitation:** An attacker could use a brute-force attack to guess user credentials or attempt to hijack a session by capturing or predicting a weak session token to perform unauthorized transfers.
- **Potential Business Impact:** **Unauthorized fund transfers** and **account takeover (ATO)**, directly resulting in financial loss for customers and the company.

3. Broken Access Control (A01)

- **How it Could Exist:** A user might be able to view or modify another customer's account or transaction history simply by changing a numeric ID in the URL parameter (an **IDOR** flaw).
- **Attacker Exploitation:** An attacker, logged in as their own account, modifies a request parameter (e.g., changing user_id=A to user_id=B) to access or modify data belonging to another user.
- **Potential Business Impact:** Unauthorized access to sensitive personal and financial data belonging to other users, resulting in privacy violations and customer distrust.

4. Cryptographic Failures (A02)

- **How it Could Exist:** The platform might store sensitive data like customer PII, account balances, or even payment card details using weak, outdated, or no encryption, or use broken hashing algorithms for passwords.
- **Attacker Exploitation:** An attacker who gains access to the underlying database can easily **decrypt or reverse-hash** the sensitive customer data if weak cryptography or outdated hashing functions (like SHA1 or MD5) were used.
- **Potential Business Impact:** **Total loss of all customer data** confidentiality, leading to significant financial and legal liability, especially concerning payment industry standards.

5. Server-Side Request Forgery (SSRF) (A10)

- **How it Could Exist:** If the platform includes a feature that fetches resources from external or internal URLs based on user input (e.g., checking a payment status from an internal service or fetching a profile image).
- **Attacker Exploitation:** An attacker provides a crafted URL pointing to an internal resource (like http://169.254.169.254/metadata in a cloud environment) or a URL for an internal API, forcing the PayX server to leak internal information or cloud credentials.
- **Potential Business Impact:** **Exposure of internal network architecture**, unauthorized access to internal APIs, and potential **theft of cloud environment credentials**.

TASK 4: SECURE DESIGN RECCOMENDATIONS (PAYX SECURE)

Risk	Recommendation
SQL Injection	Mandate Parameterized Queries/Prepared Statements for all database calls to separate data from command logic. Implement Whitelisting Input Validation to ensure user input conforms to expected data types and formats (e.g., only numbers allowed for an ID field).
Broken Access Conrol	Implement Server-Side Role-Based Access Control (RBAC) checks on every API endpoint to enforce the Principle of Least Privilege. Utilize Unpredictable, Cryptographically Strong IDs (UUIDs) instead of sequential primary keys to prevent Insecure Direct Object Reference (IDOR).
Security Misconfiguration	Employ Configuration Management Tools to enforce a hardened baseline. Disable all default accounts, unused services, and debugging interfaces in production environments. Ensure Custom Error Pages are used to prevent leakage of technical details via verbose error messages.
Vulnerable Components	Maintain a Software Bill of Materials (SBOM) and utilize Software Composition Analysis (SCA) tools to continuously scan for known vulnerabilities (CVEs) in all third-party libraries and frameworks. Establish a rapid and well-tested Patch Management Process.
SSRF	Implement a URL Whitelist to restrict resource fetching to only specific, required domains. Utilize Network Access Control Lists (NACLs) or firewall rules to prevent the application server from making any outbound requests to private/internal IP address ranges (e.g., 10.x.x.x, 192.168.x.x, and cloud metadata services).

TASK 5: REPORTING AND DOCUMENTATIONS

Subject: Lab Report on Critical Application Flaws

Assignment: OWASP Top 10 Risk Assessment (Task 3)

Testing Platform: OWASP Juice Shop

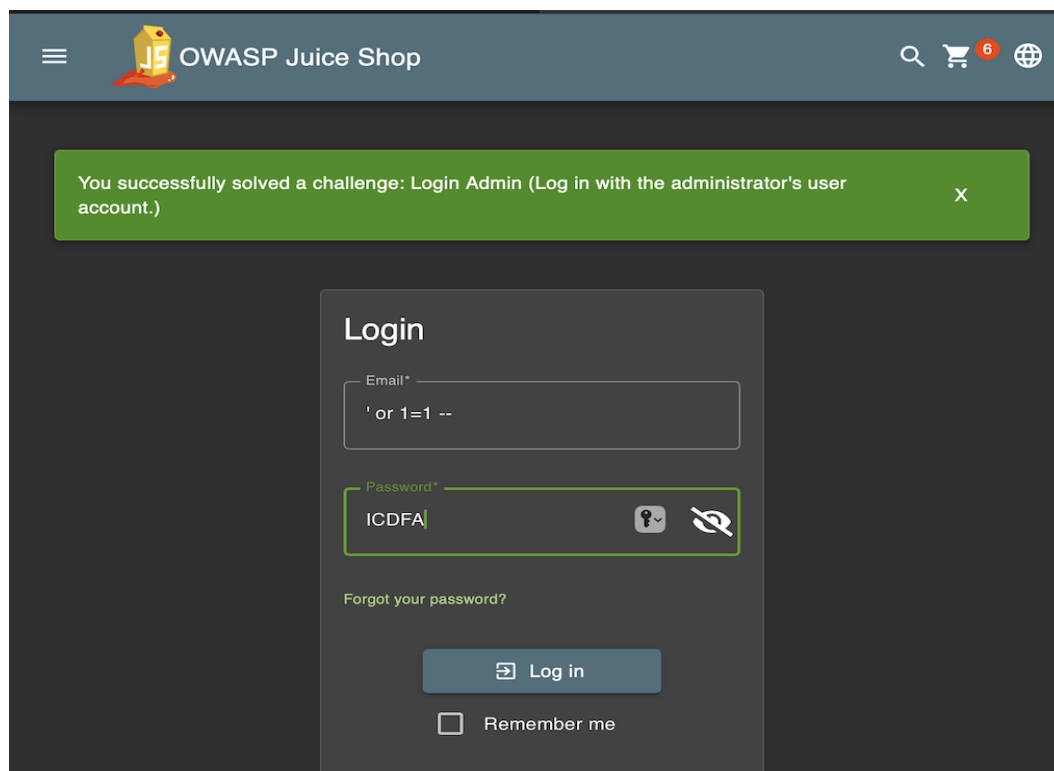
Purpose:

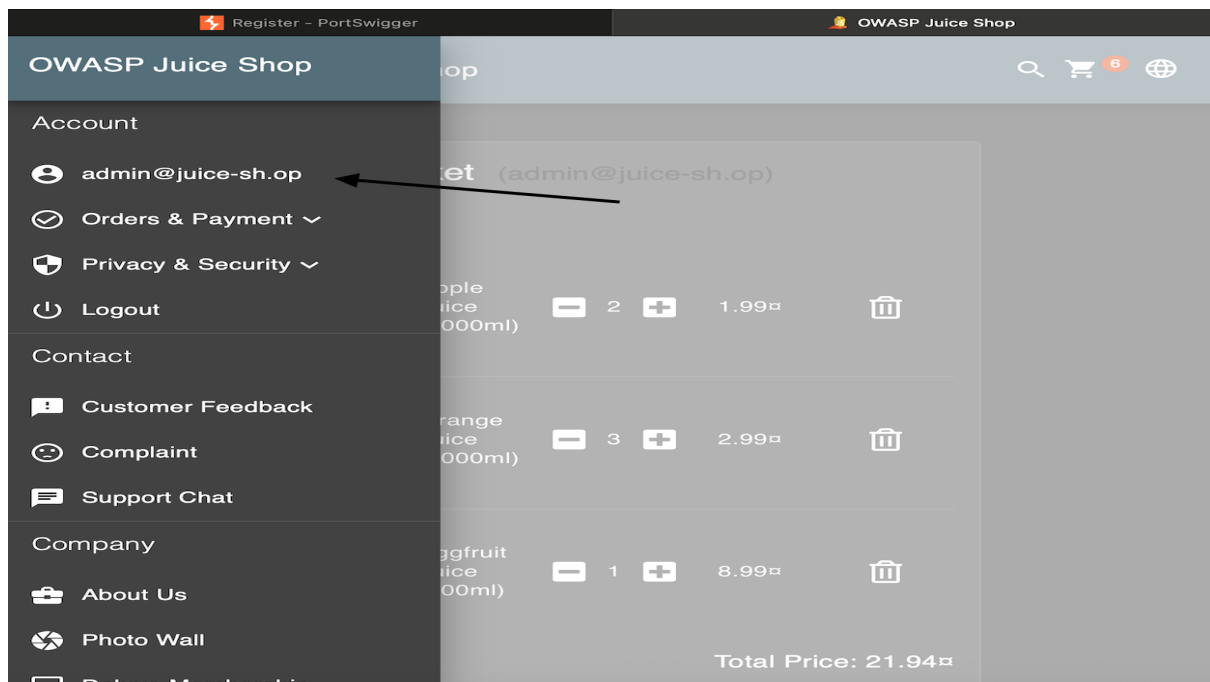
The objective of this testing phase was to execute and document proof-of-concept exploits for three critical web application vulnerabilities SQL Injection (SQLi), Cross-Site Scripting (XSS), and Broken Access Control (IDOR) within a controlled, vulnerable environment. This exercise serves to confirm the exploitability and potential impact of these flaws, providing necessary evidence for subsequent secure design recommendations.

1. Practical Test 1: SQL Injection (SQLi)

This test addresses the A03:2021 Injection category by targeting a flawed authentication implementation.

Testing Steps: The login form was targeted. The payload ' OR 1=1-- was manually entered into the Email input field. An arbitrary, invalid string was supplied for the Password field, and the form was submitted.





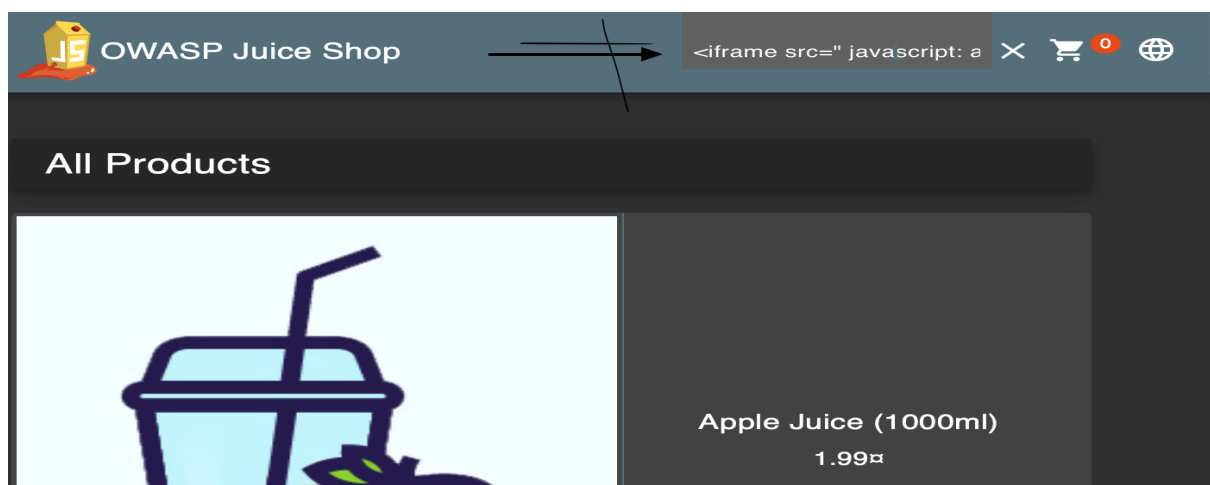
Successful authentication bypass via SQL Injection, resulting in elevated access to the system administrator account (admin@juice-sh.op).

Technical Cause: The application failed to enforce a structural separation between data and code, resulting in user-supplied input being directly concatenated into the server's SQL query string. The injected payload successfully altered the query's logic, forcing the authentication condition to resolve to a logically true state ($1=1$), while the SQL comment sequence (`--`) negated the remainder of the query, effectively bypassing the password check entirely.

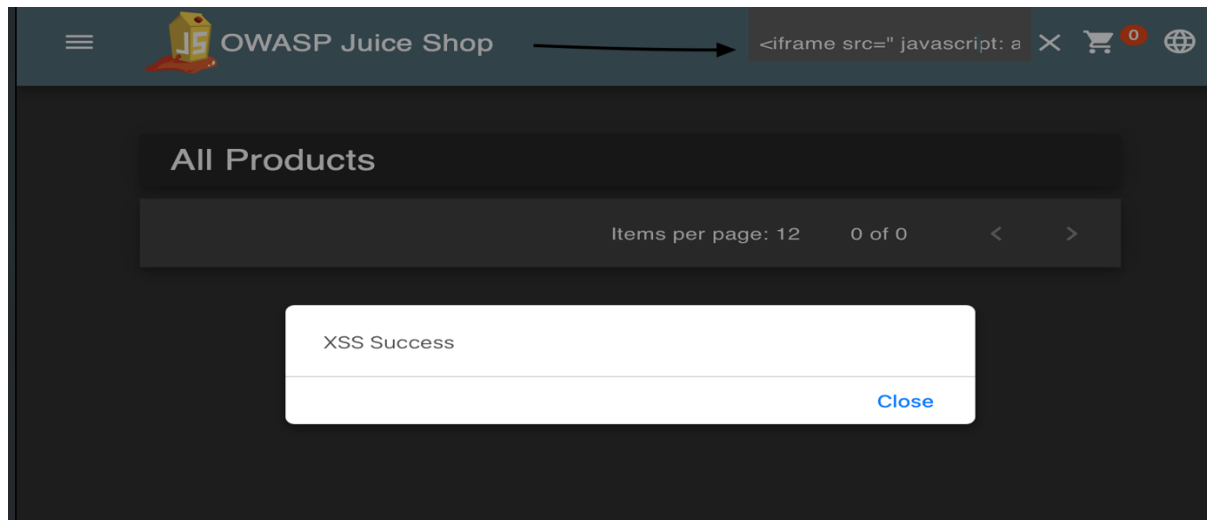
2. Practical Test 2: Cross-Site Scripting (XSS)

This test successfully demonstrated a vulnerability within the A03:2021 Injection category via a reflected vector.

Testing Steps: The application's search functionality was utilized as the injection point. The JavaScript payload `<iframe src="javascript:alert('XSS Success')">` was input into the search field and submitted.



The browser responded by executing the code, generating an unauthorized pop-up alert box.



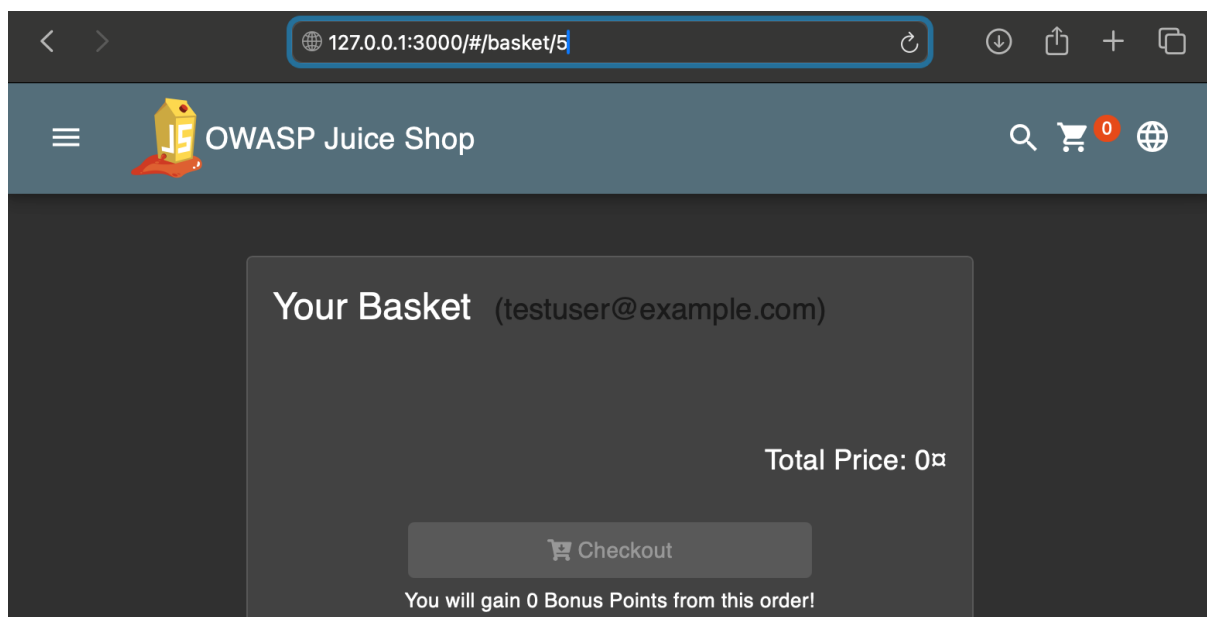
Technical Cause: The application failed to apply essential Output Encoding (HTML escaping) to the user-supplied input before it was reflected back and rendered on the client's web page. Because the browser received the malicious code unencoded, it interpreted the input as a legitimate HTML tag (`<iframe>`) and executed the embedded JavaScript.

3. Practical Test 3: Broken Access Control (IDOR)

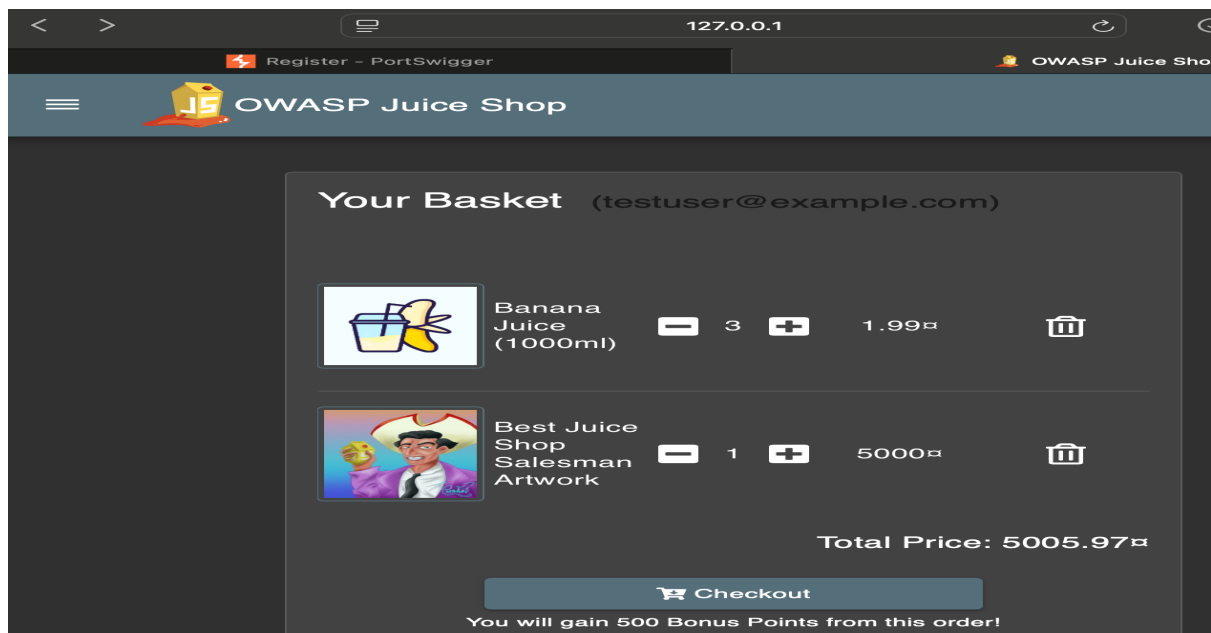
Test A: Unauthorized Basket/Cart View

This test established the existence of a vulnerability under the A01:2021 Broken Access Control category, specifically an Insecure Direct Object Reference (IDOR).

Testing Steps: I authenticated as a standard user, identified a resource identifier (Basket ID) exposed in their own basket URL (`127.0.0.1:3000/#/basket/5`).



I then manually modified the URL by substituting the valid ID with a low-value, unauthorized ID (e.g., `127.0.0.1:3000/#/basket/4`) and requested the resource. The application then successfully loaded the contents of the foreign basket.

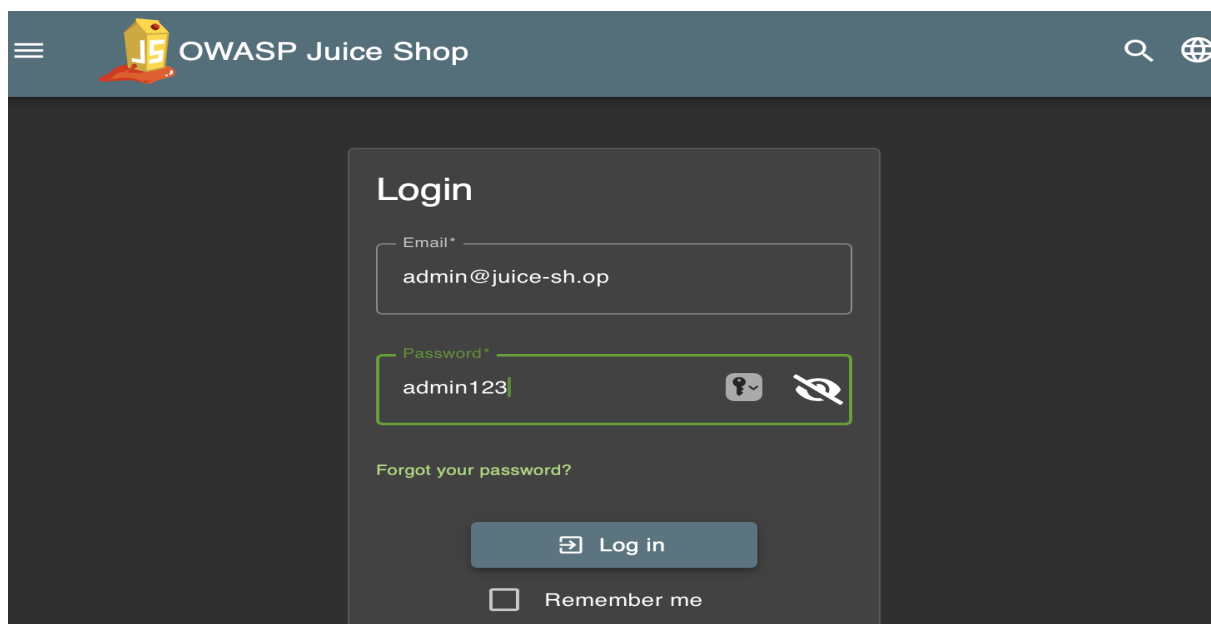


This shows a successful exploitation of an IDOR vulnerability, showing unauthorized access to another user's shopping basket (Basket ID 1) while authenticated as a different user.

Technical Cause: The system performed an incomplete security check. It verified that the user was authenticated but failed to conduct a subsequent, crucial Authorization Check (verifying the logged-in user was the rightful owner of the resource ID requested). This oversight allowed the user to bypass authorization controls by manipulating a URL parameter.

Test B: Access to Restricted Administrative Functions (Vertical Privilege Escalation)

I attempted to access a sensitive, high-privilege function or data endpoint by substituting a standard user ID with the ID or credentials associated with the **administrator account** using admin@juice-sh.op's ID to exploit the parameter that directly accepts the email address.



Technical Cause: The system performed an incomplete security check. It verified that the user was **authenticated** but failed to conduct a subsequent, crucial **Authorization Check** (verifying the logged-in user was the rightful owner of the resource ID requested). This oversight allowed the user to bypass authorization controls by manipulating a URL parameter.

SUMMARY OF RECOMMENDED ACTIONS AND IMPROVEMENTS

The successful exploitation of these three vulnerabilities highlights foundational weaknesses in the application's security posture. To transition the application to a more secure state, the following comprehensive and immediate remedial actions are mandated:

1. **Injection Prevention (SQLi):** Implement a mandatory policy to use Prepared Statements (Parameterized Queries) for all database queries. This approach ensures that user input is always treated as data, fundamentally preventing code injection. Additionally, enforce strict Input Validation (allow-listing) for all user-controllable fields.
2. **Cross-Site Scripting (XSS) Mitigation:** Apply universal Output Encoding to all dynamic data sourced from user input before it is rendered to the client's browser. This neutralizes executable characters, causing them to be displayed as literal text instead of running as code.
3. **Access Control Enforcement (IDOR):** Implement a robust, server-side authorization check. Every request to a resource with a user-specific identifier must include a step that verifies the authenticated user ID is the authorized owner of the resource being requested. This ensures that a user cannot access another person's data simply by guessing a URL parameter.

REFERENCES

BBC News. (2017, July 28). Bupa fined £175,000 for data protection breach. Retrieved from <https://www.bbc.com/news/business-40748118>

CISA. (2021). Remediating the SolarWinds and Related Attacks. Retrieved from [https://www.cisa.gov/us-cert/sites/default/files/publications/Remediating SolarWinds and Related Attacks.pdf](https://www.cisa.gov/us-cert/sites/default/files/publications/Remediating_SolarWinds_and_Related_Attacks.pdf)

CISA. (2022, September 14). Software Bill of Materials (SBOM) and Cyber Supply Chain Risk Management. Retrieved from <https://www.cisa.gov/resources-tools/resources/software-bill-materials-sbom-and-cyber-supply-chain-risk-management>

Cloudflare. (2019, July 23). Cloudflare's response to the AWS IAM bug. Retrieved from <https://blog.cloudflare.com/cloudflares-response-to-the-aws-iam-bug/>

O'Keeffe, J. (2017). Cloudbleed: Cloudflare's Memory Leak. Security Boulevard. Retrieved from <https://securityboulevard.com/2017/02/cloudbleed-cloudflares-memory-leak/>

OWASP Foundation. (2021). OWASP Top 10 - 2021: The Ten Most Critical Web Application Security Risks. Retrieved from <https://owasp.org/Top10/>

OWASP Foundation. (2023a). SQL Injection Prevention Cheat Sheet. Retrieved from [https://cheatsheetseries.owasp.org/cheatsheets/SQL Injection Prevention Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html)

OWASP Foundation. (2023b). Cross Site Scripting Prevention Cheat Sheet. Retrieved from [https://cheatsheetseries.owasp.org/cheatsheets/Cross Site Scripting Prevention Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html)

OWASP Foundation. (2023c). Access Control Cheat Sheet. Retrieved from [https://cheatsheetseries.owasp.org/cheatsheets/Access Control Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html)

OWASP Foundation. (2023d). Security Misconfiguration Prevention Cheat Sheet. Retrieved from [https://cheatsheetseries.owasp.org/cheatsheets/Security Misconfiguration Prevention Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Security_Misconfiguration_Prevention_Cheat_Sheet.html)

OWASP Foundation. (2023e). Server Side Request Forgery Prevention Cheat Sheet. Retrieved from [https://cheatsheetseries.owasp.org/cheatsheets/Server Side Request Forgery Prevention Cheat Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html)

SANS Institute. (2015). The Sony Pictures Attack. Retrieved from <https://www.sans.org/reading-room/whitepapers/casestudies/sony-pictures-attack-35805>

SANS Institute. (2020). Capital One Data Breach - Case Study. Retrieved from <https://www.sans.org/reading-room/whitepapers/casestudies/capital-one-data-breach-case-study-39405>

Security Intelligence. (2017, June 28). LinkedIn Hacking Incident: The 2012 Breach and Lessons Learned. Retrieved from <https://securityintelligence.com/posts/linkedin-hacking-incident-the-2012-breach-and-lessons-learned/>

Vance, P., & Cardenas, A. A. (2017). The Equifax Data Breach: A Case Study in Security Failures. IEEE Security & Privacy Magazine, 15(6), 1-5.